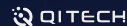


• BOOTCAMP QI TECH 2026 · DIA 2 · MANHÃ

Contratos, RFC e REST

Recapitulação das camadas, a RFC da entrega, e os contratos: HTTP, status e REST.



Os **professores** do dia



Gilberto Oliveira

Administração e custódia



Gabriel Orsi

Financeiro Tech



João Bebber

LaaS

Pergunta pode ser feita a qualquer momento, e quem está pelo Meet pergunta pelo chat.

O DIA DE HOJE

Cronograma

MANHÃ · 10:00 ÀS 12:00

01 **Recapitulação: as camadas**

O caminho da requisição e uma camada nova: connectors

02 **A RFC**

O documento que explica o sistema e defende cada regra

03 **Contratos, HTTP e REST**

Cliente e servidor, métodos, status, nomes de rota e identificadores

TARDE · 13:30 ÀS 16:30

04 **Banco de dados**

Relacional, ACID e modelagem: entidades, estados e relações

05 **Fluxos**

O fluxo feliz, o alternativo e o de erro

De manhã vocês ganham o vocabulário do contrato; de tarde, o que decide o contrato. O bloco 02 explica por quê.

O ROTEIRO

Duas horas com **quatro blocos**

10:00**Recapitulação**

As camadas da API (Application Programming Interface) e uma que vocês ainda não viram: connectors

10:20**A RFC**

O documento da entrega, com um exemplo aberto na tela

10:50**Contratos e HTTP**

Cliente, servidor, métodos, status e a RFC 9110

11:20**REST e identificadores**

Nomes de rota, exemplos bons e ruins, id e UUID

Pergunta pode ser feita a qualquer momento e quem está pelo Meet pergunta pelo chat.

BLOCO 01

01 O que ficou de **sábado** **passado**

As camadas do base-service, o restaurante de sempre, e uma peça nova:
os connectors.

10:00 às 10:20

RECAPITULAÇÃO

A requisição, **camada por camada**

IDA · O PEDIDO



VOLTA · A RESPOSTA

A seta anda num sentido só: **resources** → **controllers** → **repositories** → **models**. A novidade de hoje sai do controller para o lado: quando o pedido precisa de alguém de fora, quem liga é o **connector**.

As camadas, **em uma linha cada**

CAMADA	PERGUNTA QUE RESPONDE	O QUE NÃO FAZ
middlewares/	O que vale para toda requisição?	conhecer uma rota específica
schemas/	Esse JSON tem a forma certa?	falar com banco, decidir regra
resources/	Como isso entra e sai em HTTP?	if de negócio, SQL
controllers/	Pode ou não pode?	escrever consulta, saber que existe HTTP
repositories/	Como busco e guardo no banco?	decidir se aquilo era permitido
models/	Como a tabela é?	ter regra dentro
dtos/	O que sai, com que nome?	buscar no banco, decidir regra
connectors/	Como falo com um serviço de fora?	decidir regra, falar com o nosso banco

Connectors: quando o pedido depende de alguém de fora

O QUE É

A camada que conversa com um serviço de outra empresa ou de outro time. Guarda três coisas: o endereço, o tempo máximo de espera e o desembrulho da resposta.

ANALOGIA

O telefone que a cozinha usa para ligar para o fornecedor. Quem disca é o chef, nunca o garçom. O telefone não decide se compra e não entra na despensa: ele fala com quem está fora e traz o que ouviu.

REGRA DE OURO

Connector não decide e não toca no nosso banco. Se apareceu um `if` de negócio dentro dele, a regra vazou do controller. E **timeout é obrigatório**: quem não responde não pode segurar a nossa requisição para sempre.

QUEM CHAMA QUEM

`resource` → `controller` → `connector` → serviço de fora

O `resource` nunca chama um `connector`: quem decide perguntar lá fora é a regra de negócio.

O QUE CUSTA

Você passou a depender de algo que não controla. Quando o serviço de fora cai ou demora, alguém decide o que responder ao seu cliente — e quem decide é o `controller`, com um erro seu em `src/errors/`.

EXEMPLO DE IMPLEMENTAÇÃO

Um **connector**, e onde ele entraria no projeto de vocês

- `src/connectors/bureau.py`

```
class BureauConnector:

    BASE_URL = environ["BUREAU_URL"]
    TIMEOUT = 5                                # quem está fora sempre tem prazo

    def get_document_status(self, document_number: str) -> dict:
        url = f"{self.BASE_URL}/documents/{document_number}"
        response = requests.get(url, timeout=self.TIMEOUT)
        return response.json()                  # desembrulha e devolve; não decide nada
```

No projeto de vocês, um connector entraria se a conta precisasse de uma consulta externa antes de abrir: antifraude, bureau de crédito, consulta de documento. O CPF que o base-service já valida em `src/utls/document_number.py` não precisa de connector: ele se resolve dentro de casa. **Se ninguém liga para fora, a pasta fica vazia.**

BLOCO 02

02 A RFC

O documento que explica o sistema e, principalmente, defende cada regra que ele tem.

10:20 às 10:50

A RFC

Na RFC o **banco** vem primeiro. Na aula, o **HTTP**.

NA RFC: BANCO PRIMEIRO

O contrato é consequência do modelo. Se depósito e transferência são a mesma tabela com uma coluna `type`, a rota é uma só. Se são duas tabelas, são duas rotas, dois corpos e dois status.

O endereço sai do desenho das entidades. Quem escolhe a rota antes de escolher a tabela acaba com uma rota que o banco não consegue atender.

NA AULA: HTTP PRIMEIRO

Para discutir se a rota é uma ou são duas, você precisa saber o que é uma rota, o que é um método e o que é um contrato. Sem esse vocabulário, a discussão de modelagem não acontece.

Então: a manhã ensina **como o contrato se escreve**.
A tarde ensina **o que decide o contrato**.

Quando vocês sentarem para escrever a RFC, invertam de volta: desenhem as tabelas e só depois escrevam as rotas.

RFC: a decisão técnica com o raciocínio junto

O QUE É

RFC (Request for Comments) é um documento que explica uma decisão técnica e o raciocínio por trás dela. O nome vem da internet: o próprio HTTP que vocês vão usar hoje é uma RFC.

ANALOGIA

A receita da casa com a margem escrita. A receita diz o que fazer; a margem diz por que é assim e o que acontece se você trocar um ingrediente.

REGRA DE OURO

Escreva para alguém que **não está na sua cabeça** e não vai poder te perguntar nada.

O QUE A RFC DE VOCÊS PRECISA TER

- **Explicar o sistema** que o grupo construiu.
- **Mostrar o ecossistema.** Mais de um serviço, um worker, um banco separado: cada parte aparece em detalhe.
- **Explicar a regra de negócio:** o que o sistema permite, o que ele barra e por quê.

ENTREGA

PDF, até 13/10 (slides ou documento, a lógica é que não é livre)

Quatro seções, **nesta ordem**

01

Banco de dados

Quais entidades existem, o que cada uma guarda e como uma aponta para a outra.

As entidades definidas aqui determinam o que vem depois.

02

Endpoints e contratos

Cada rota, com método, o que entra, o que sai e qual status.

Sai direto do desenho das entidades, nunca o contrário.

03

Fluxos

O feliz, o alternativo e o de erro. O que acontece quando falta saldo, quando a conta está encerrada, quando o serviço de fora não responde.

Assunto da tarde.

04

Decisões e trade-offs

O que vocês escolheram, o que descartaram e o que isso custou.

É a seção em que cada escolha aparece com o seu motivo.

A RFC vai **do que existe para o que acontece**: primeiro as entidades, depois as rotas, depois os fluxos que passam por elas.

BLOCO 03

03 Contratos entre cliente e servidor

Quem pede, quem responde, em que língua, e como se dá nome às coisas.

10:50 às 12:00

CONTRATOS

QUEM PEDE E QUEM RESPONDE

Cliente e servidor: um papel, não uma máquina

O QUE É

Cliente é quem pede. Servidor é quem responde. É um papel na conversa, não um tipo de computador: a API de vocês é servidora do aplicativo e cliente do bureau no mesmo segundo.

ANALOGIA

O restaurante de sempre. Quem senta na mesa pede; a cozinha responde. Quando o restaurante liga para o fornecedor, ele vira cliente de outra pessoa sem deixar de ser restaurante.

IDEIA-CHAVE

Quem começa a conversa é sempre o cliente. O servidor não liga de volta sozinho. O custo: para avisar de algo que aconteceu depois, alguém precisa perguntar de novo — ou entra uma fila, que o base-service não tem.

O MESMO SISTEMA, TRÊS CONVERSAS

App do banco
(cliente)

GET /account/{key}

(servidor) A API de
vocês

A API de vocês
(agora cliente)

GET /document/{cpf}

(servidor) O bureau,
via connector

Seu navegador
(cliente)

GET /doc/html/rfc9110

(servidor)
datatracker.ietf.org

A CONVERSA, POR ESCRITO

Uma **requisição** e uma **resposta**, lado a lado

IDA · MÉTODO, CAMINHO, CABEÇALHOS, CORPO

```
POST /accounts HTTP/1.1
Host: api.seubanco.com
Content-Type: application/json
```

```
{
  "customer_key": "9f1c8e2a-...",
  "type": "checking"
}
```

VOLTA · STATUS, CABEÇALHOS, CORPO

```
HTTP/1.1 201 Created
Content-Type: application/json
```

```
{ "account_key": "3b7e41d0-..." } # do DTO
```

```
HTTP/1.1 404 Not Found
```

```
{ "title": "Customer not Found",
  "description": "...", "translation": "...",
  "code": "QIT001001" } # de errors/
```

Os dois corpos viajam em JSON (JavaScript Object Notation). Na ida, o cliente escolhe método, caminho e corpo; na volta, o servidor escolhe o status e o corpo. **O corpo de sucesso sai do DTO; o corpo de erro tem sempre os mesmos quatro campos** — title, description, translation e code — para o cliente conseguir tratar erro sem adivinhar.

Os **métodos**^[*]: a intenção do pedido

GET

Buscar dados

Ler. **Nunca muda nada.**

Pode repetir mil vezes: a resposta é a mesma e nada acontece.

POST

Criar novo recurso

Criar algo que ainda não existe.
Responde 201.

Repetiu, criou de novo.
Dois cliques, duas transferências.

PUT

Substituir

Substituir o recurso todo. Você manda todos os campos.

Esqueceu um campo?
Ele vai embora.

PATCH

Atualizar

Mudar um pedaço.
Manda só o que mudou.

Mais gentil, mas exige dizer no contrato o que pode ser ajustado.

DELETE

Remover

Remover. Em banco, quase sempre vira mudança de estado.

Apagar linha de dinheiro remove o histórico: encerre a conta, em vez de deletar.

O método carrega a **intenção**; o caminho carrega a **coisa**. Um GET que debita uma conta é repetido pelo navegador e por robôs de busca, porque os dois foram feitos acreditando que GET não muda nada. [*] Estes cinco são os que vocês vão usar. A RFC 9110 define outros — HEAD, OPTIONS, TRACE, CONNECT — fora do escopo de hoje.

Os **status**: como foi o pedido

2xx

Sucesso

200 achei e devolvi
201 criei — e devolvo a key
204 fiz, e não tenho o
que devolver

Sucesso é o cliente poder
seguir sem ler o corpo para
saber se deu certo.

3xx

Redirecionamento

301 mudou de endereço,
para sempre
304 não mudou: usa o que
você tem

Raro numa API interna.
Aparece quando entra cache
ou o endereço muda.

4xx

Erro de regra de negócio

400 corpo torto — parou
no schema
403 sem INTERNAL-TOKEN
404 essa key não existe
409 o estado não permite

A culpa é do pedido. Reenviar
igual dá o mesmo resultado.

5xx

Erro interno

500 nós falhamos
503 não estamos de
pé agora

O pedido estava certo. Aqui o
cliente pode tentar de novo —
e vocês têm um bug.

A faixa é a primeira coisa que o cliente lê, e muitas vezes a única. Devolver 200 com um erro escondido no corpo obriga todo mundo a ler o corpo para saber se deu certo e alguém vai esquecer. No base-service, o status de sucesso é escolhido no **resource**; o de erro vem junto com o erro, de `src/errors/`.

200 com mensagem de erro no corpo

ASSIM NÃO

HTTP/1.1 **200 OK**

Content-Type: application/json

```
{
  "success": false,
  "message": "Saldo insuficiente"
}
```

A faixa diz que deu certo. O corpo diz que não deu. O cliente só descobre se ler o JSON e conhecer o campo `success` — que é invencionice nossa, não do HTTP.

ASSIM

HTTP/1.1 **422 Unprocessable**

Content-Type: application/json

```
{
  "title": "Insufficient Balance",
  "description": "...",
  "translation": "...",
  "code": "QIT002014"
}
```

A faixa já entrega o resultado, e o corpo de erro tem os mesmos quatro campos de sempre, vindos de `errors/`.

O custo do 200 com erro aparece do outro lado: o monitoramento conta 100% de sucesso, o *retry* automático nunca dispara, e cada cliente novo precisa ser avisado de que 200 aqui não quer dizer 200.

RFC 9110: onde “o que significa um 201” está escrito

- datatracker.ietf.org/doc/html/rfc9110

Internet Engineering Task Force (IETF)
Request for Comments: [9110](#)
STD: 97
Obsoletes: [2818](#), [7230](#), [7231](#), [7232](#), [7233](#), [7235](#),
[7538](#), [7615](#), [7694](#)
Updates: [3864](#)
Category: Standards Track
Published: June 2022
ISSN: 2070-1721

R. Fielding, Ed.
Adobe
M. Nottingham, Ed.
Fastly
J. Reschke, Ed.
greenbytes

HTTP Semantics

Abstract

The Hypertext Transfer Protocol (HTTP) is a stateless application-level protocol for distributed, collaborative, hypertext information systems. This document describes the overall architecture of HTTP, establishes common terminology, and defines aspects of the protocol that are shared by all versions. In this definition are core protocol elements, extensibility mechanisms, and the “http” and “https” Uniform Resource Identifier (URI) schemes.

This document updates RFC 3864 and obsoletes RFCs 2818, 7231, 7232, 7233, 7235, 7538, 7615, 7694, and portions of 7230.

Status of This Memo

This is an Internet Standards Track document.

This document is a product of the Internet Engineering Task Force (IETF). It represents the consensus of the IETF community. It has received public review and has been approved for publication by the Internet Engineering Steering Group (IESG). Further information on Internet Standards is available in Section 2 of RFC 7841.

Information about the current status of this document, any errata, and how to provide feedback on it may be obtained at <https://www.rfc-editor.org/info/rfc9110>.

O QUE É

HTTP não é convenção de time nem gosto de framework: é um documento público. A RFC 9110, *HTTP Semantics*, de junho de 2022, define os métodos, os status e o que cada um significa, para HTTP/1.1, HTTP/2 e HTTP/3.

POR QUE ELA IMPORTA PARA VOCÊS

Quando dois times discutem se aquilo é 400 ou 422, a discussão termina aqui. É a mesma razão pela qual a RFC de vocês existe, na escala do sistema de vocês: um lugar onde o combinado está escrito e qualquer um pode conferir.

DETALHE QUE VALE SABER

Ela substituiu a RFC 7231, que ainda aparece em muito tutorial antigo. Documento vivo também envelhece e a RFC diz qual ela aposentou.

O CARDÁPIO ORGANIZADO POR PRATOS

REST: o endereço nomeia uma coisa

O QUE É

REST (Representational State Transfer) é uma convenção para nomear o que a API expõe. O endereço nomeia **uma coisa**: um recurso, substantivo, em inglês. O que você quer fazer com ela vem do método, nunca do endereço.

ANALOGIA

O cardápio organizado por pratos, não por ações. Não existe a linha "trazerLasanha": existe *lasanha*, e pedir ou trocar é o verbo que você usa com o garçom.

REGRA DE OURO

Se o endereço tem verbo, ele está fora da convenção.

`/create_account` e

`/get_account` são o mesmo

NO BASE-SERVICE

```
application.add_api_route("/sample_entity", ..., methods=["POST"])
application.add_api_route("/sample_entity/{sample_entity_key}", ...)
application.add_api_route("/sample_entities", ...) # a lista
```

Repare: singular no item, plural na lista. A convenção mais comum é plural nos dois. **Escolham uma** e escrevam na RFC qual foi.

O QUE CUSTA

REST é convenção, não lei, e existe ação que não vira substantivo confortável: um estorno, um fechamento de conta. Quando não couber, escolham o substantivo mais próximo (`/transactions`) e escrevam o porquê.

O MESMO SISTEMA, DOIS CARDÁPIOS

Exemplos **bons** e exemplos **ruins**

ASSIM

POST /account

cria uma conta · 201 com a key

GET /account/{account_key}

lê uma conta · 200 ou 404

PATCH /account/{account_key}

ajusta um campo dela

POST /account/{account_key}/transaction

movimenta: o filho pertence à conta

GET /account/{account_key}/transactions?
limit=20&page=1

extrato paginado · filtro na query string

ASSIM NÃO

~~POST /createAccount~~

o verbo já está no método; tirem do endereço

~~GET /account/delete/42~~

GET que muda estado, e id sequencial na URL

~~GET /account/{key}/statement~~

"statement" é relatório, não recurso: o recurso é *transaction*

~~GET /contas/{chave}~~

português no meio do inglês — escolham uma língua

~~POST /account/{key}/transferMoney~~

a ação virou rota; o recurso é *transaction*

Cada linha da esquerda tem o seu par na direita. Substantivo, em inglês, identificado pela key; filtro, ordenação e página vão na **query string**, nunca no caminho.

Um **depósito** e uma **transferência** são a mesma coisa?

Antes da rota, responda no banco

Um **tipo** ou duas **entidades**?

DESENHO A

Uma entidade, com uma coluna *type*

```
transaction
  type: DEPOSIT | TRANSFER | FEE
  origin_account_id      nulo no depósito
  destination_account_id nulo no saque
```

POST /account/{key}/transaction

Ganha — o extrato é uma consulta só, e a tarifa é mais uma linha

Paga — colunas que só valem para um tipo ficam nuláveis, e "transferência tem origem e destino" vira `if` no controller, não restrição no banco

DESENHO B

Uma entidade por tipo

deposit	transfer
account_id	origin_account_id
amount	destination_account_id
	amount

POST /account/{key}/deposit

POST /transfer

Ganha — cada tabela só tem coluna obrigatória: o banco recusa o impossível sem código

Paga — o extrato vira união de duas consultas, e toda regra nova se escreve duas vezes

Os dois desenhos são válidos. O que a RFC precisa registrar é a **resposta defendida**. É esta página que a banca vai abrir na RFC de vocês e a tarde de hoje é sobre como decidi-la.

O IDENTIFICADOR EXPOSTO NA URL

GET /account/1 funcionou. Então /account/2 também.

RASPAR A BASE

Contar de 1 em 1

Um id sequencial é previsível: um script simples percorre a base inteira sem precisar adivinhar nada.

Se faltar uma checagem de permissão em uma rota, ela vaza tudo.

CONTAR O NEGÓCIO

Entregar o número

Quem abre conta hoje e recebe o id 8.412 sabe quantas contas vocês abriram até hoje.

Repete amanhã e descobre a taxa de crescimento do banco de vocês.

NÃO É UNIVERSAL

O 42 de todo mundo

O id 42 só significa algo dentro daquela tabela, naquele banco.

Na hora de conversar com outro sistema, dois "42" diferentes se encontram e ninguém sabe de quem é qual.

O id está fazendo exatamente o que ele foi feito para fazer. **O erro foi mostrá-lo para o mundo de fora.**

DUAS CHAVES, DOIS TRABALHOS

id por dentro, **UUID** por fora

O QUE É

O id inteiro é interno: é ele que as chaves estrangeiras usam, e ele nunca sai da API.

O ..._key é um UUID — 128 bits em 36 caracteres e é ele que aparece na URL.

ANALOGIA

O número de matrícula interno e o número do protocolo que o cliente leva no papel. O banco se organiza pelo primeiro; o cliente só conhece o segundo.

IDEIA-CHAVE, E O QUE CUSTA

UUID não é adivinhável e é **universal**: dá para gerar um aqui e outro no parceiro, sem combinar nada e sem colidir. Em troca, são duas colunas, e o UUID indexa pior que um inteiro por isso as junções

• `src/models/account.py`

```
class Account(Base):
    __tablename__ = "account"

    id = Column(Integer, primary_key=True)           # interno: as relações usam este
    account_key = Column(CHAR(36), nullable=False)   # externo: é o que sai na URL
    customer_id = Column(Integer, ForeignKey(Customer.id), nullable=False)

    __table_args__ = (UniqueConstraint("account_key"),)
```

EXEMPLO DE IMPLEMENTAÇÃO

A **key** na rota, o **id** no banco

- `src/app.py` · `src/repositories/account_repository.py`

```
# src/app.py – toda rota de item usa a key, nunca o id
application.add_api_route("/account/{account_key}", account_resource.on_get_by_key, methods=["GET"])
application.add_api_route("/account/{account_key}", account_resource.on_patch, methods=["PATCH"])

# src/repositories/account_repository.py
def create(self, account_data: dict) -> Account:
    account = Account()
    account.account_key = str(uuid4())           # o UUID nasce aqui
    self.session.add(account)
    return account

def get_by_key(self, account_key: str) -> Account:
    return self.session.query(Account).filter(Account.account_key == account_key).first()
```

O mundo de fora só conhece a key. O repository traduz key em objeto uma vez, e a partir daí o resto do código usa o id. Repare que quem gera o UUID é o repository, no create não o cliente, e não o banco.

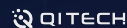
Cada conceito, **uma pergunta**

CONCEITO	PERGUNTA QUE RESPONDE	O QUE NÃO FAZ
cliente / servidor	Quem pede e quem responde?	o servidor não começa a conversa
HTTP	Em que língua os dois falam?	não diz quais são as suas rotas
método	Qual é a intenção do pedido?	não identifica o recurso
status	Como foi?	não explica o caso (isso é o corpo do erro)
REST	Como dou nome ao que exponho?	não é lei, nem define o formato do corpo
schema / DTO	Qual a forma do que entra e do que sai?	não decide se pode
..._key (UUID)	Como o mundo de fora aponta para esta linha?	não é a chave usada nas junções
RFC	Por que é assim?	não substitui o código

• BOOTCAMP QI TECH 2026 · DIA 2 · TARDE

Banco de dados e fluxos

Relacional e ACID, modelagem de entidades, estados e relações, e os fluxos da requisição.



Três horas com **quatro assuntos**

13:30	Banco de dados Relacional e ACID, depois modelagem: chaves, estados e relações
15:00	Fluxos Regra de negócio, fluxo feliz, o alternativo e o de erro
15:40	RFC Mostramos como funciona a RFC na prática
16:10	Próximos passos O que fazer até a próxima aula e onde pedir socorro

De manhã vocês viram **como o contrato se escreve**. Esta tarde é sobre **o que decide o contrato**.

BLOCO 01

01 Banco de dados

Como o relacional funciona, o que ACID garante, e como se decide o que vira tabela.

13:30 às 15:00

BANCO DE DADOS

O QUE HÁ DENTRO DO BANCO

Banco relacional: uma planilha com regras

O QUE É

Um jeito de guardar dados em tabelas com colunas de tipo fixo, em que uma linha aponta para outra por uma chave. Quem descreve a tabela em Python é `src/models/`; quem a cria de verdade é `database/database.sql`, escrito em SQL (Structured Query Language).

ANALOGIA

Uma planilha, com regras. Cada **aba** é uma tabela, cada **linha** é um registro, e a chave primária é a coluna que identifica aquela linha. A diferença é que aqui uma célula pode **referenciar uma linha de outra aba**, e a planilha se recusa a apontar para linha que não existe.

REGRA DE OURO

Célula que referencia uma linha inexistente em outra aba, o banco **recusa**. O banco é a última camada capaz de recusar o dado inválido, mesmo quando o código deixa passar.

O VOCABULÁRIO, LADO A LADO

A aba → `table · account`

A linha → `row · uma conta`

A coluna que a identifica → `primary key · id`

Por que **relacional** quando o assunto é dinheiro

01

Tipo fixo

A coluna diz o que cabe nela. Dinheiro não vira texto no meio do caminho, e data não vira número.

02

Restrição

NOT NULL, UNIQUE, chave estrangeira: o banco recusa o impossível sem depender de nenhum i.f.

03

Transação

Várias escritas viram uma só: ou todas valem, ou nenhuma vale. É o assunto do próximo slide.

04

Consulta

Somar, agrupar, filtrar e paginar anos de extrato é exatamente o trabalho para o qual ele foi feito.

O QUE A TRANSFERÊNCIA PRECISA

ACID: quatro garantias, uma transferência

O caso de todos os dias: alguém transfere R\$ 200 para outra conta e paga R\$ 2 de tarifa. São **três escritas**, e as quatro letras abaixo são o que garante que elas valham como uma coisa só.

A Atomicidade

Ou debita, credita e cobra a tarifa, ou nada disso acontece. As três escritas são uma operação só.

Sem ela: o dinheiro sai de uma conta e não chega na outra.

C Consistência

O banco nunca aceita um estado que quebra as regras declaradas nele: conta sem cliente, key repetida, valor nulo onde não pode.

Sem ela: a ficha aponta para uma gaveta vazia, e ninguém percebe até o extrato.

I Isolamento

Duas transações na mesma conta não enxergam o rascunho uma da outra. Cada uma vê um estado inteiro, nunca um pela metade.

Sem ela: é o saldo do slide anterior: 100 lido duas vezes, 160 gastos.

D Durabilidade

Depois do commit, está gravado. Vale queda de energia, queda do processo e reinício do container.

Sem ela: faltou luz e o depósito sumiu, mas o comprovante do cliente não sumiu.

Vocês não implementam nenhuma delas: vocês decidem onde a transação começa e onde ela termina.

ATOMICIDADE, NO CÓDIGO DE VOCÊS

Três escritas, **um commit**

- `src/controllers/transaction_controller.py`

```
def transfer(self, transfer_data: dict) -> dict:
    origin = self.account_repository.get_by_key(transfer_data["origin_account_key"])
    total = transfer_data["amount"] + FEE

    if origin.balance < total:                                # recusa ANTES de qualquer escrita
        raise InsufficientBalance(origin.account_key)

    self.transaction_repository.create_transaction(origin, -total)
    self.transaction_repository.create_transaction(destination, transfer_data["amount"])
    self.session.commit()                                     # as três viram uma só
```

Enquanto o `commit` não roda, tudo isso é rascunho na sessão. Se a segunda escrita explodir, o middleware de sessão dá `rollback` e **nada aconteceu** nem o débito, nem a tarifa. É por isso que o `commit` é a última linha antes do `return`, e por isso que ele mora no controller: só ele sabe que o trabalho terminou.

Isolamento: dois caixas, a mesma conta

O QUE É

Enquanto uma transação está no meio do trabalho, as outras não enxergam o rascunho dela. O banco decide o que fazer com a segunda: fazê-la esperar, deixá-la ler o estado antigo ou recusá-la.

ANALOGIA

Dois caixas atendendo a mesma conta. Enquanto um está com a ficha na mão, o outro não lê o saldo pela metade ele espera a ficha voltar para a gaveta.

IDEIA-CHAVE, E O QUE CUSTA

Isolamento tem vários níveis: quanto mais forte, mais uma transação espera pela outra. Hoje basta saber que ele existe e que o padrão do Postgres já impede ler o rascunho da outra.

NO PROJETO DE VOCÊS

Se o saldo mora numa **coluna**, duas transferências simultâneas são o caso que vocês precisam ter pensado e a RFC diz como resolveram.

Se o saldo é **somado das transações**, o problema muda de lugar: cada movimento é uma linha nova, e a leitura fica mais cara.

REGRA DE OURO

Leu para decidir e depois escreveu com base no que leu? **Esse par precisa acontecer dentro da mesma transação.**

Travar a **linha**: uma transação por vez

O QUE É

O banco oferece a possibilidade de **travar uma linha**. Quem pegou a trava mexe naquela conta; quem chegou depois espera na fila até o `commit`. Com isso só uma transação por vez acontece naquela linha.

ANALOGIA

A ficha da conta sai da gaveta e fica na mão de um caixa só. O segundo caixa não lê uma cópia: ele espera a ficha voltar.

NO CÓDIGO

```
SELECT balance FROM account
WHERE id = 42
FOR UPDATE;    -- trava esta linha# SQLAlchemy
session.query(Account)\
    .filter(Account.id == 42)\
    .with_for_update().first()
```

O QUE CUSTA

Fila. Duas operações na mesma conta deixam de ser simultâneas, e travar na ordem errada em duas contas trava as duas para sempre trave sempre na mesma ordem.

SUBTÓPICO 02

02 Modelagem

O que vira tabela, em que estados uma entidade existe, e como uma aponta para a outra.

Três perguntas, **nesta ordem**

01

O que é entidade?

Uma coisa que existe por si, que você precisa encontrar depois e cujo histórico importa.

Cliente é. Conta é. Transação é.
"Tarifa" pode ser entidade ou só uma coluna e essa é uma decisão de vocês.

02

Em que estado ela está?

Quais estados existem, quem pode virar o quê, e se o sistema precisa lembrar por onde ela passou.

É aqui que entram os enumeradores e os eventos.

03

Como uma aponta para a outra?

1:1, N:1 e N:N. Onde mora a chave estrangeira, e quando aparece uma terceira tabela.

Um relacionamento errado só aparece quando o dado já existe, e corrigi-lo exige migração.

Toda resposta aqui vira uma linha na seção 2 da RFC e cada rota que vocês escreveram de manhã é consequência de uma delas.

A TABELA, DO LADO DO BANCO

id carrega a relação, **key** sai na URL

- database/database.sql

```
CREATE TABLE account (
    id          SERIAL PRIMARY KEY,           -- interno: carrega a relação
    account_key CHAR(36) NOT NULL UNIQUE,     -- externo: sai na URL
    customer_id INTEGER NOT NULL REFERENCES customer(id),
    status_id   INTEGER NOT NULL REFERENCES account_status(id),
    balance     BIGINT NOT NULL DEFAULT 0,    -- centavos, inteiro
    created_at  TIMESTAMP NOT NULL DEFAULT now(),
    updated_at  TIMESTAMP NOT NULL DEFAULT now(),
    CONSTRAINT balance_nao_negativo CHECK (balance >= 0) -- o banco recusa saldo negativo
);
```

Quatro decisões numa tabela só: a chave que o mundo vê (`account_key`), a que o banco usa (`id`), o dono (`customer_id`, obrigatório conta sem cliente não nasce) e o estado. Mexeu neste arquivo, lembre: ele roda **uma vez, quando o banco nasce**.

Status: um enumerador é uma tabela

O QUE É

Um estado com nome fixo (PENDING, ACTIVE, CLOSED) que vive numa tabela própria. A entidade guarda só um `status_id` apontando para lá: é o `SampleEntityStatus` do `base-service`.

ANALOGIA

A lista de status pregada na parede da dispensa. A ficha não escreve o estado à mão: aponta para uma linha da lista. Ninguém escreve “ativoo” com dois o, e ninguém inventa estado que não existe.

IDEIA-CHAVE, E O QUE CUSTA

O banco passa a recusar estado inexistente isso é **restrição no banco, sem if**. Em troca, são uma tabela e uma junção a mais em toda leitura, e a ordem do `models/__init__.py` passa a importar: o status entra *antes* da entidade que o usa.

• `src/models/account.py` • `src/dtos/account_dto.py`

```
class Account(Base):
    __tablename__ = "account"

    status_id = Column(Integer, ForeignKey(AccountStatus.id), nullable=False)
    status = relationship("AccountStatus", foreign_keys=[status_id],
                          lazy="selectin")

# no DTO, o número vira texto – e só aí
dto["status"] = account.status.enumerator
```

Na tabela o status é um número; na resposta, uma palavra. Quem faz a travessia é o DTO.

A CONTA ESTÁ BLOQUEADA. DESDE QUANDO?

Eventos: uma linha por mudança de estado

O PROBLEMA

A coluna `status_id` responde **onde** a conta está. Não responde desde quando, por quem, nem quantas vezes isso já aconteceu. Quando alguém pergunta, a resposta não existe mais: foi sobrescrita.

O QUE É

Uma segunda tabela, com uma linha por transição: de que estado, para que estado, quando e por quê. A entidade continua com o estado atual; o caminho fica ao lado.

ANALOGIA, E O QUE CUSTA

A ficha mostra onde o pedido está; os carimbos no verso mostram por onde ele passou. O preço é uma escrita a mais em toda transição: a coluna é a verdade do agora, a tabela de eventos é a verdade do que aconteceu.

- `database/database.sql`

```
CREATE TABLE account_status_event (  
  id SERIAL PRIMARY KEY,  
  account_id INTEGER NOT NULL REFERENCES account(id),  
  from_status_id INTEGER REFERENCES account_status(id), -- nulo ao nascer  
  to_status_id INTEGER NOT NULL REFERENCES account_status(id),  
  reason VARCHAR(255),  
  created_at TIMESTAMP NOT NULL DEFAULT now()  
);
```

Em sistema financeiro, o histórico de estados costuma ser exigência regulatória.

QUEM PODE VIRAR O QUÊ

Estados: a lista mora no banco, a transição mora no controller



Este desenho é linear de propósito: no sistema de vocês, **BLOCKED** provavelmente volta para **ACTIVE** e essa seta também é uma decisão de vocês. **Estado final é final**, e é ele que responde 409: conta encerrada não recebe transação.

As três formas de **relação**

N:1 · A MAIS COMUM

Muitas contas, um cliente

A chave estrangeira mora do lado do *muitos*: `account.customer_id`.

Quase toda linha do sistema de vocês tem uma dessas. É o formato padrão.

1:1 · A RARA

Uma linha, uma linha

Separar um pedaço da entidade: dados sensíveis ou raramente lidos numa tabela ao lado.

Custa uma junção para guardar o que caberia na mesma tabela. Use quando o pedaço é opcional, sensível ou pesado.

N:N · A QUE ENGANA

Vários de cada lado

Uma conta conjunta com dois titulares; um cliente com várias contas.

Não existe coluna que guarde isso: **exige uma terceira tabela.**

Para descobrir onde a chave estrangeira mora, leia a frase nos dois sentidos: “uma conta tem **um** cliente” e “um cliente tem **várias** contas”. O lado do *um* vira coluna no lado do *vários*. **Estas três formas vivem no código, não no banco.** O banco só garante que a chave aponta para uma linha que existe; que a relação é N:1, 1:1 ou N:N, e o que ela significa no negócio, quem declara é `src/models/` e quem decide é o controller.

N:N NA PRÁTICA · FATURA DE CARTÃO

Um pagamento quita várias faturas;
uma fatura aceita **vários pagamentos**



O cliente paga R\$ 800 e manda quitar duas faturas atrasadas; no mês seguinte paga a mesma fatura em duas parcelas. Nenhum dos dois casos cabe numa coluna: a tabela do meio guarda o que **só existe na relação** — amount, quanto daquele pagamento foi para aquela fatura. Sem o UNIQUE do par, o mesmo pagamento entra duas vezes na mesma fatura e o total fecha errado.

Cada peça, uma pergunta

PEÇA	PERGUNTA QUE RESPONDE	O QUE NÃO FAZ
tabela (model)	Como este dado fica guardado?	não decide regra nem formata resposta
id	Como o banco aponta para esta linha?	não sai na API
..._key (UUID)	Como o mundo de fora aponta para ela?	não é usado nas junções
chave estrangeira	De quem é esta linha?	não barra regra de negócio, só o impossível
enumerador (status)	Em que estado isto está agora?	não conta como chegou aqui
tabela de eventos	Por onde isto passou?	não é a verdade do estado atual
transação (ACID)	Estas escritas valem juntas?	não decide se elas eram permitidas
tabela de ligação	Quem se relaciona com quem, em N:N?	não guarda dado que é só de um lado

FECHAMENTO DO BLOCO

O que este bloco **cobriu**

- 01 Relacional e ACID: o commit define onde a operação começa e termina.
- 02 Chaves: o id carrega a relação, a key sai na URL.
- 03 Estados: a lista mora no banco, a transição mora no controller.
- 04 Relações: N:1, 1:1 e N:N, e onde mora a chave estrangeira.

BLOCO 03

03 Fluxos da requisição

A sequência de passos entre receber a requisição e responder: o caminho feliz, o alternativo e o de erro.

15:00 às 16:10

FLUXOS

A PONTA DE ENTRADA

```
POST /client  
  
{  
  "name": "Ana Souza",  
  "document_number": "12345678901",  
  "birth_date": "1994-03-12"  
}
```

O MIOLO



O que o seu serviço faz, em que ordem, e o que ele responde quando algo não dá certo.

O ARMAZENAMENTO

```
database.sql  
  
CREATE TABLE client (  
  id          BIGSERIAL PRIMARY KEY,  
  client_key  CHAR(36) NOT NULL UNIQUE,  
  name       TEXT NOT NULL,  
  document_number CHAR(14) NOT NULL,  
  birth_date  DATE NOT NULL,  
  created_at  TIMESTAMPTZ NOT NULL  
);
```

A rota veio do bloco 1, a tabela veio do bloco 2. O bloco de hoje é inteiro sobre a caixa do meio.

A PONTA DE ENTRADA

```
POST /client  
  
{  
  "name": "Ana Souza",  
  "document_number": "12345678901",  
  "birth_date": "1994-03-12"  
}
```

O MIOLO

- 1 Verificar se o CPF passado é válido
- 2 Verificar se já existe um cliente com mesmo CPF
- 3 Verificar idade do cliente
- 4 Criar cliente
- 5 Atualizar status do cliente para "aprovado"
- 6 Retornar

O ARMAZENAMENTO

```
database.sql  
  
CREATE TABLE client (  
  id          BIGSERIAL PRIMARY KEY,  
  client_key  CHAR(36) NOT NULL UNIQUE,  
  name        TEXT NOT NULL,  
  document_number CHAR(14) NOT NULL,  
  birth_date  DATE NOT NULL,  
  created_at  TIMESTAMPTZ NOT NULL  
);
```

Seis passos, e cada um deles com uma forma de dar errado. É por isso que este bloco existe.

Fluxo: a sequência entre receber e responder

O QUE É

A sequência de passos entre receber a requisição e responder.

O fluxo mora no **controller**: é ele que ordena os passos, aplica as regras e decide o que será escrito.

ANALOGIA

A ficha de pedido na cozinha. O garçom trouxe o pedido e vai levar o prato; o que acontece entre os dois, e em que ordem, é a ficha.

Se faltou um ingrediente, a ficha diz o que a cozinha faz.

REGRA DE OURO

Um fluxo só está pronto quando o **caminho de erro** também está desenhado.



Por que desenhar **antes** de escrever

01

Desenho é barato de mudar

Apagar uma seta custa três segundos. Reescrever um controller custa uma tarde.

02

Te ajuda a detalhar as etapas

Passo a passo: qual validação é necessária e qual vai ser o tratamento dela.

Quais são os status das entidades, e em que momento eles mudam.

03

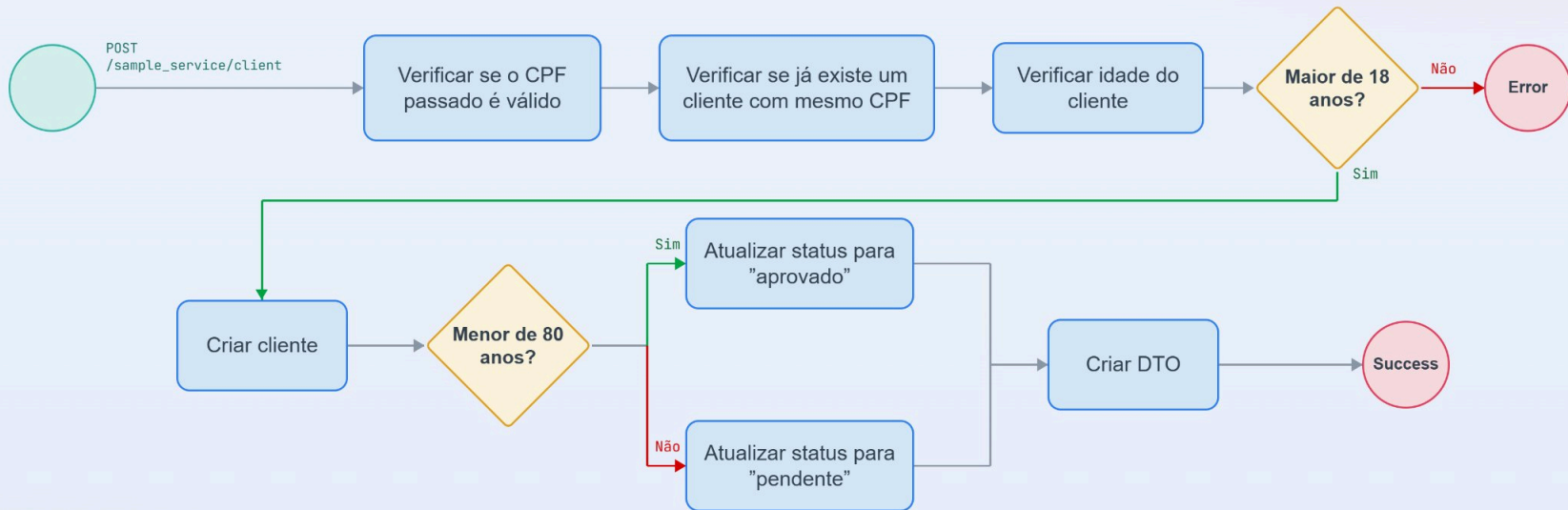
Defender a solução fica mais fácil

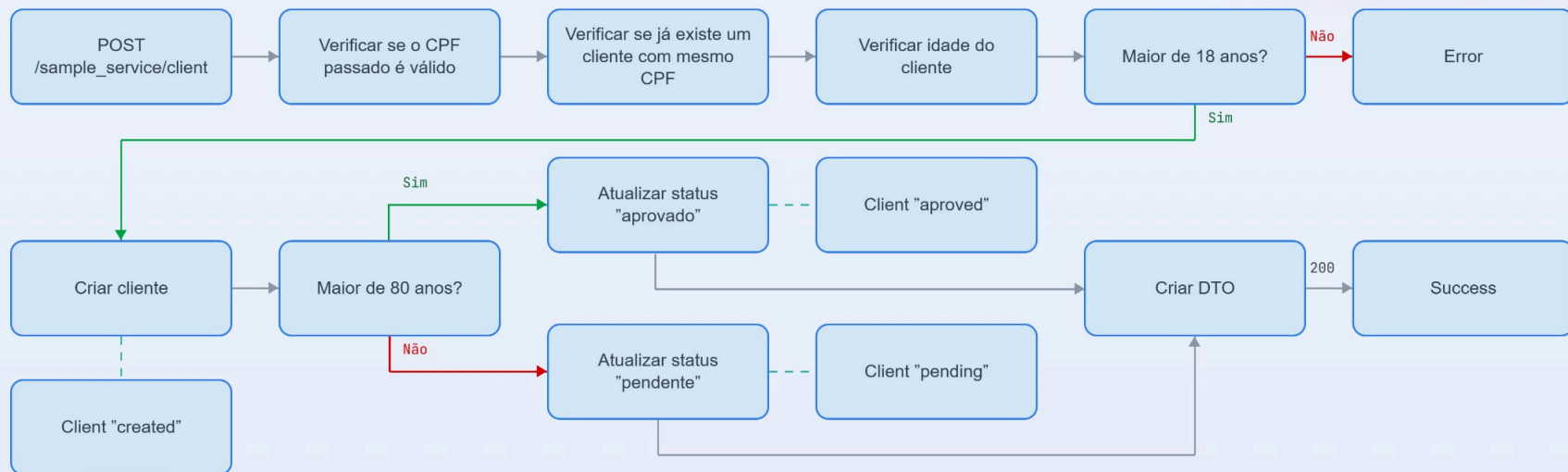
Com o fluxo na tela, você aponta o ramo e mostra a decisão que tomou ali.

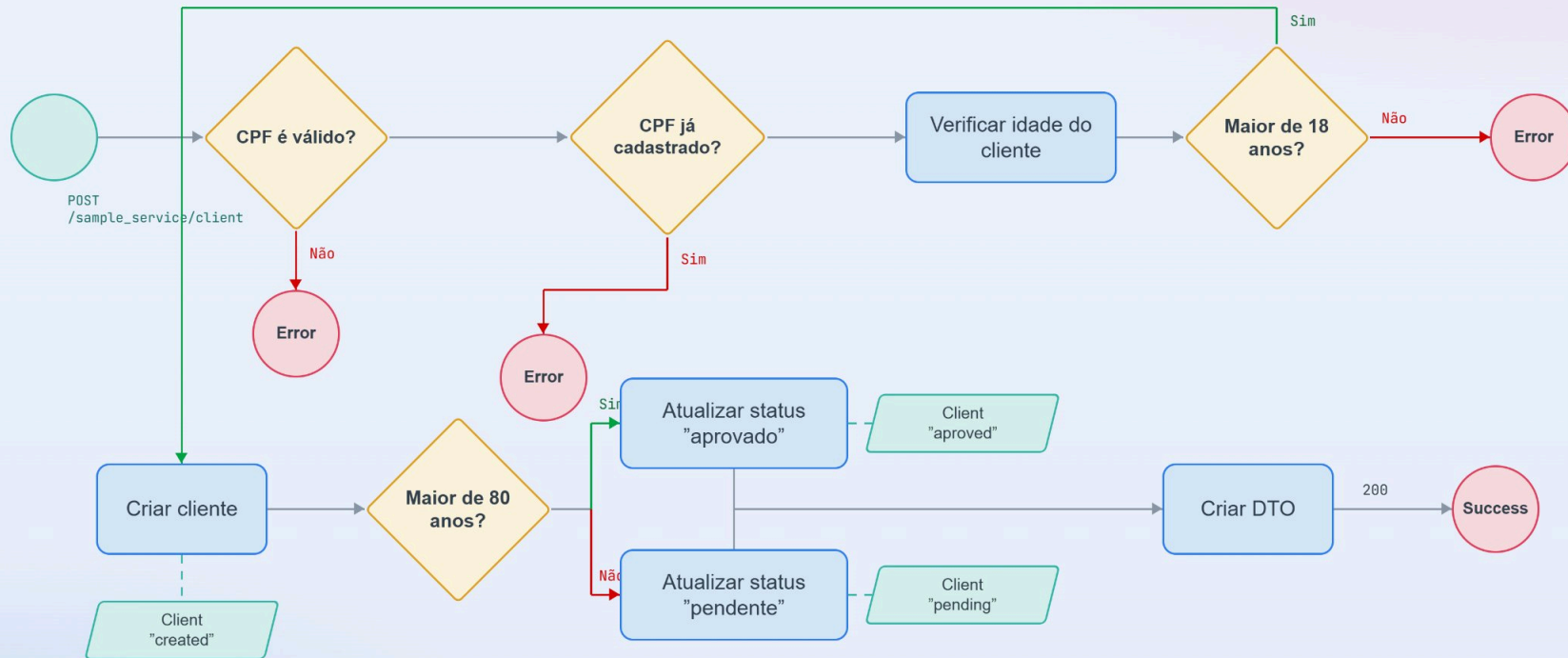
Tão importante quanto desenhar uma boa solução é conseguir mostrar que ela é boa.

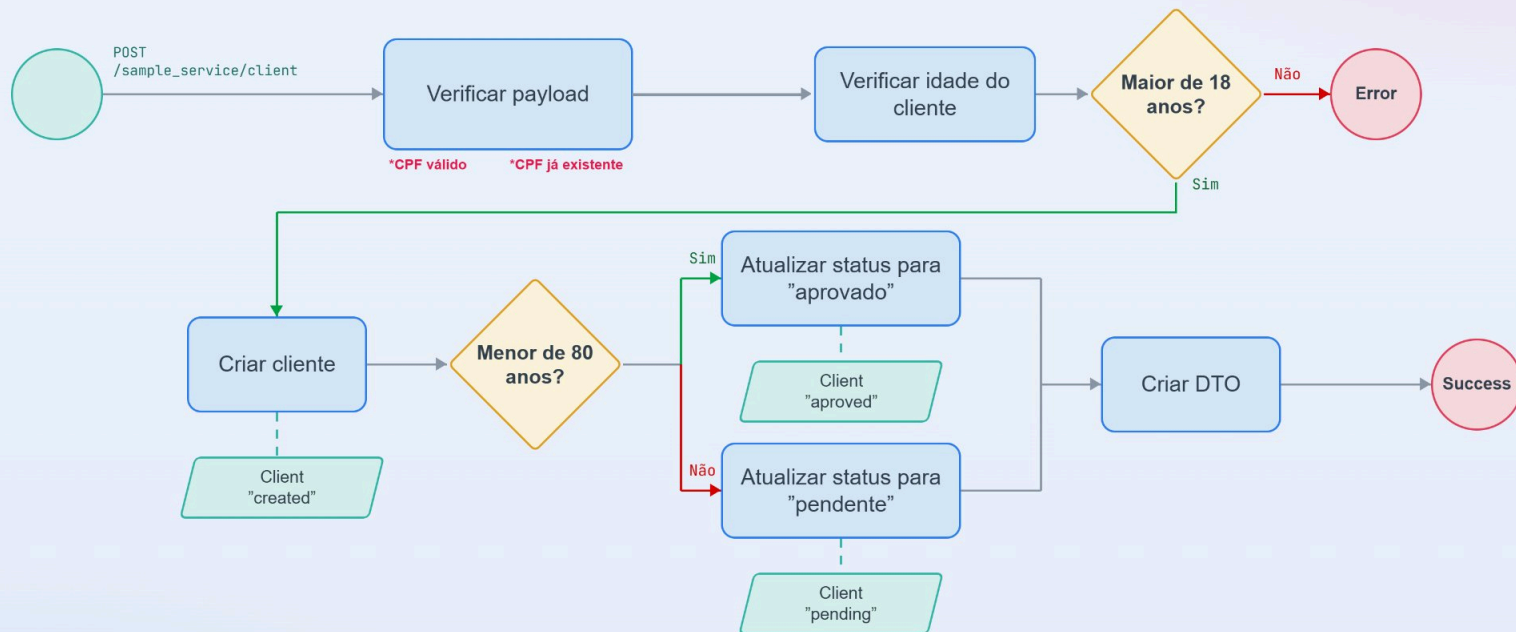


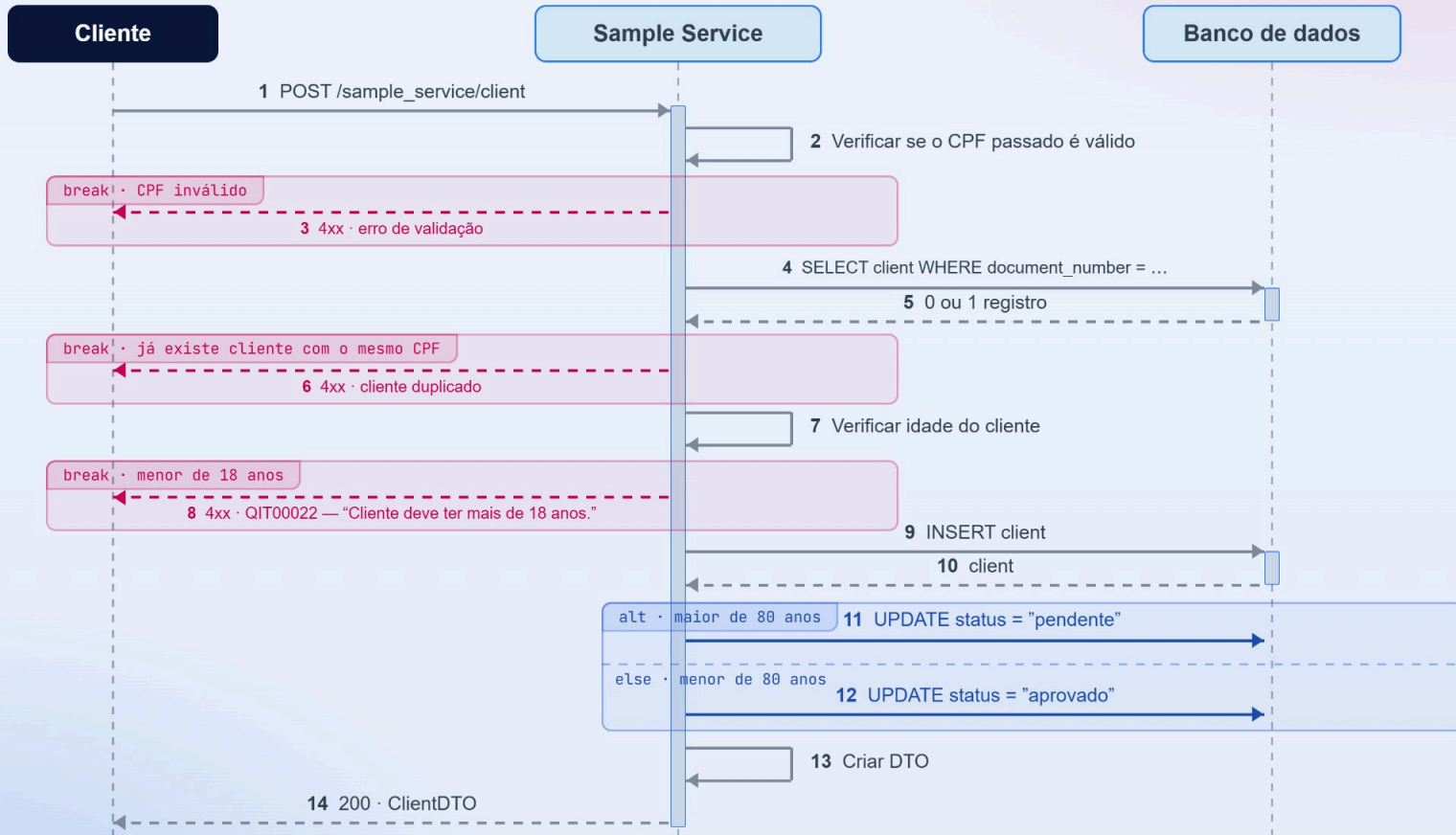
Criação de **cliente**, do pedido à resposta











Seis regras para um **bom fluxo**

01

Ser objetivo

Um verbo por caixa. Se você precisa de um "e" para descrever o que ela faz, são duas caixas.

02

Tem que ser sequencial

Uma caixa só começa quando a anterior termina, e todo caminho chega a um fim.

03

Ser claro

Sim e não escritos na seta, e o texto da caixa no idioma de quem lê. Losango sem rótulo não é pergunta.

04

Captar os três caminhos

O que dá certo atravessa reto; alternativos e exceções saem pelo lado. Quem olha três segundos acha o caminho feliz.

05

Demonstrar estados

Cada caixa que grava diz em que estado a entidade fica. O desenho termina onde o ciclo dela termina.

06

Ter nomes do projeto

Rotas, campos e status saem do código que existe. Inventar nome de status sobrevive à revisão e chega na implementação.

O teste: entregue o desenho para alguém que não estava aqui. Se a pessoa souber o que acontece quando dá errado, está bom.

O que entra e o que sai de **/account**

- **POST /sample_service/client/{client_key}/account**

```
{  
  "account_type": "checking",  
  "branch": "0001"  
}
```

```
// 201
```

```
{  
  "account_key": "7b1e..."  
}
```

A CHAVE ESTÁ NO CAMINHO

O `client_key` é endereço, não dado do corpo. Isso muda a resposta quando ele não existe: 404, e não 422.

A CONTA NÃO EXISTE SOZINHA

Ela depende de um cliente que precisa existir antes. É o passo que a criação de cliente não tinha.

Cinco perguntas que o seu desenho precisa **responder**

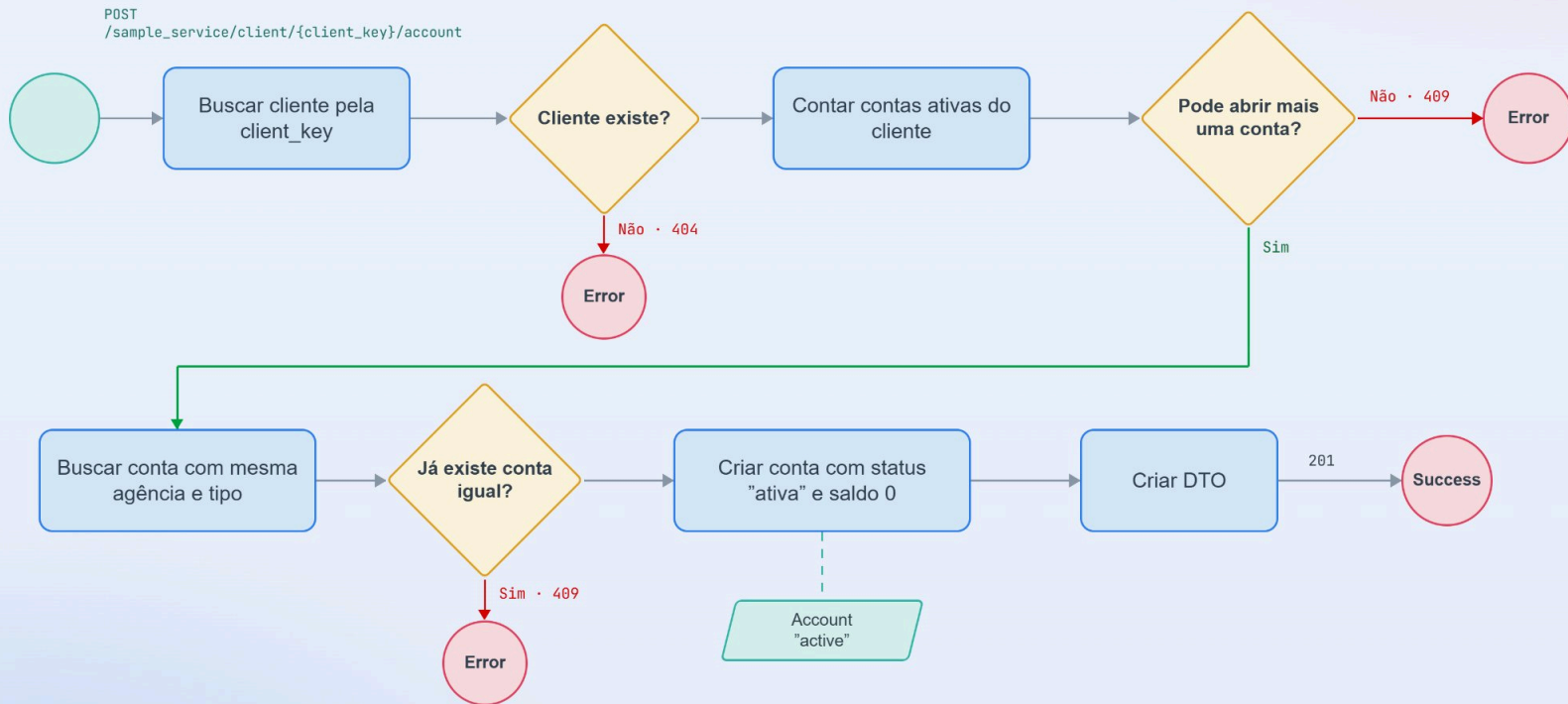
1. O cliente dessa chave existe? E se não existir, o que volta?
2. Esse cliente pode abrir mais uma conta? Quantas ele já tem?
3. Em que estado a conta nasce, e quem decide isso?
4. Onde mora o saldo: numa coluna da conta ou na soma das transações?
5. Quem precisa ser avisado quando a conta é criada?

A REGRA QUE MAIS SOME

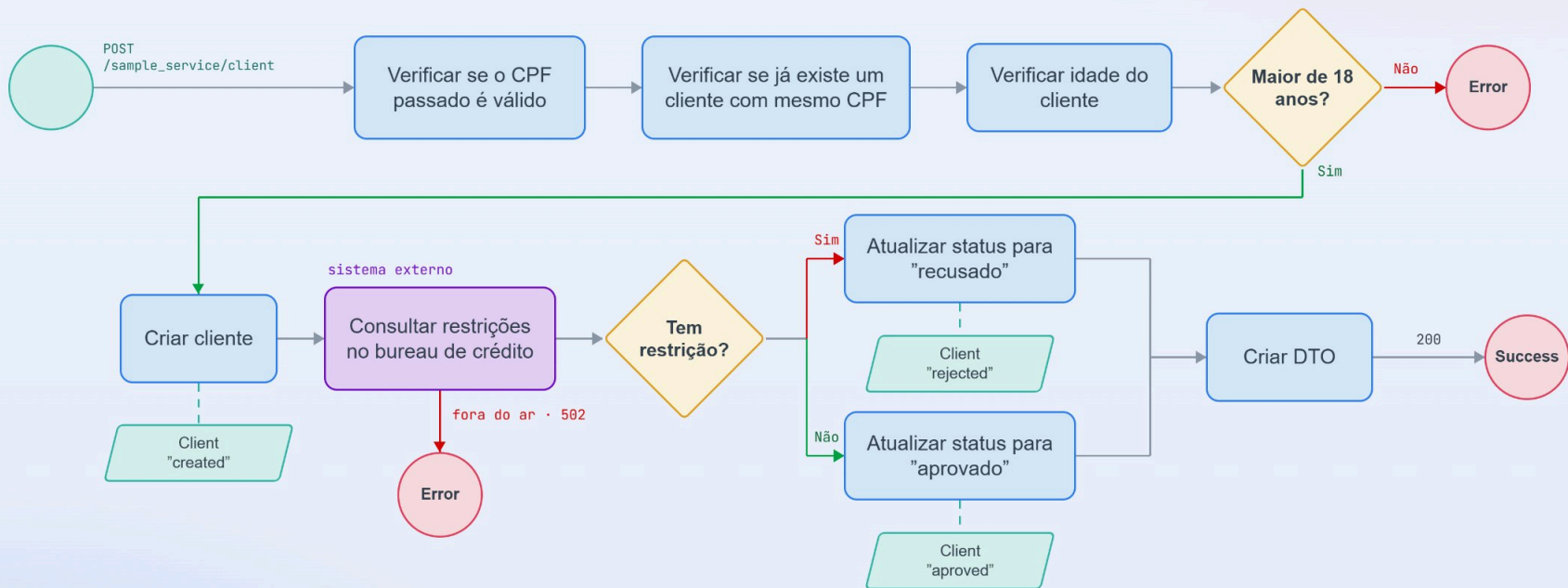
Quase todo desenho tem o "o cliente existe?". Poucos têm o "ele pode abrir mais uma?".

É uma regra de negócio, não um erro técnico, e por isso ela não aparece sozinha.

As seis perguntas, **respondidas no desenho**



E se o cliente precisar passar por uma **análise de crédito**?



Uma caixa nova trouxe dois ramos novos: o cliente com restrição, que você previu, e o bureau fora do ar, que não depende de você.

Agora sim, os **códigos**

2xx

Principal ou alternativo

Conseguiu o que queria. Está dentro de um caminho que você desenhou.

4xx

Exceção que você desenhou

Não conseguiu, e sabe por quê. O ramo existe no desenho e o corpo do erro explica o caso.

5xx

O que ninguém desenhou

Não conseguiu, e ninguém sabe por quê. Todo 5xx é um ramo que faltou no fluxo.

Agora a rota de vocês: **POST /client**

- **POST /client**

```
{
  "name": "Ana Souza",
  "document_number": "12345678901",
  "birth_date": "1994-03-12"
}

// 201
{
  "client_key": "9c2f...",
  "name": "Ana Souza",
  "created_at": "2026-09-19T13:02:11Z"
}
```

TRINTA SEGUNDOS AQUI

Não vamos discutir nome de campo. Isto é exemplo, não gabarito.

REPARE NO QUE SAI

A resposta devolve `client_key`, e não o `id` do banco.

O QUE NÃO É LIVRE

Os campos são de vocês. Ter decidido, e escrito na RFC, não é.

BLOCO 04 DE 4

04 RFC

Como uma decisão vira documento, e o documento vira discussão.

16:30 às 17:00

RFC

Vocês passaram o dia tomando decisões.
Quantas delas alguém conseguiria
contestar amanhã?

Contestar exige saber o que você considerou e o que cada alternativa custava

RFC: alguém defende uma solução por escrito

O QUE É

O documento em que alguém **defende uma sugestão de solução** para o problema que está resolvendo.

O QUE ELA FAZ

Contextualiza quem vai ler, expõe a parte difícil e descreve como a solução se implementa.

REGRA DE OURO

RFC é proposta, não registro. Ela existe para alguém **discordar** antes de o código existir.

O que uma RFC **carrega**

01

Contexto para quem vai ler

O mínimo que a pessoa precisa saber para acompanhar o resto. Nem tudo o que você aprendeu chegando até aqui.

02

Qual é a parte difícil

O ponto do problema que resiste. Se ele fosse fácil, a RFC não precisaria existir.

03

Como a solução se implementa

O que muda, onde, e em que ordem. Sem isso, ninguém consegue avaliar o custo.

Nenhuma das três é seção obrigatória. São informações que precisam estar no documento, e o arranjo depende do problema.

Três regras para a opção que você **não** escolheu

01

Mostre o preço real

Se a alternativa descartada só tem defeitos, ela foi construída como espantalho.

Se você não consegue escrever uma frase que a faça parecer razoável, provavelmente não a entendeu.

02

Diga quando ela ganharia

Toda opção descartada é a melhor em algum cenário. Dizer qual é esse cenário prova que você entendeu o que estava descartando.

03

Inclua “não fazer nada”

É quase sempre a opção mais barata e a mais legítima, e precisa ser descartada explicitamente.

Se você não consegue explicar por que o estado atual não serve, vale reconsiderar se a mudança é necessária.

Hoje vocês aprenderam **na ordem contrária**

COMO O DIA FOI DADO

- 1 Rotas e contratos
- 2 Banco de dados
- 3 Fluxos

De fora para dentro. HTTP e REST moldam como você pensa o resto, e vocês ainda não tinham isso.

COMO A CASA LÊ UMA RFC

- 1 O banco
- 2 Os fluxos
- 3 As rotas

De dentro para fora. O banco é o estado do sistema; a rota é consequência de quem já sabe o que guarda.

Agora vocês sabem por quê. Quando abrirem a RFC de alguém, comecem pelo banco.

O que cada bloco de hoje **vira** na RFC

BLOCO	O QUE VOCÊS DECIDIRAM HOJE	ONDE ISSO ENTRA NA RFC
Bloco 1	verbo, rota e formato do corpo	o contrato, na implementação: o que quebra para quem já integra
Bloco 2	o que é coluna, o que é tabela, o que é constraint	o desenho da solução, e quase sempre a parte difícil
Bloco 3	os ramos, e o código de cada falha	a implementação concreta, e a lista de erros que o cliente da API precisa tratar

Uma RFC do desafio não tem conteúdo novo. Ela tem as decisões destes três blocos, escritas para alguém discordar.

O que o autor precisa que o time **decida**

A PERGUNTA

Abrir o enum do schema para os quatro status, ou mantê-lo estreito?

Registrar o evento de created, ou aceitar que a trilha começa em pending?

Renomear a classe SampleEntityFinalStatus?

O QUE O AUTOR DEFENDE

Abrir, mas a outra posição é legítima, e o custo recai sobre quem integra.

Registrar: uma trilha que omite o começo induz quem a lê a uma conclusão errada.

Sem preferência forte, e está aqui porque, não perguntado, vira discussão no code review.

Perguntas concretas, com as opções na mesa. "Alguma consideração?" não recebe resposta.

Como saber que a sua RFC virou **formulário**

O SINTOMA

Uma seção com um parágrafo que não diz nada, só para não ficar vazia

As alternativas descartadas são obviamente ruins

O texto está no passado: "foi decidido", "implementamos"

Nenhum número, nenhum caminho de arquivo, nenhum trecho de código

A seção existe porque "toda RFC tem"

O QUE FAZER

apague a seção

ou você não entendeu as alternativas, ou não havia alternativa: diga isso

reescreva no presente da proposta, ou admita que é registro e mude o título

a parte difícil está afirmada, não provada

nenhuma seção é obrigatória: apague e veja se o documento perde alguma coisa

17:00 · FECHO DO DIA

Bora pra cima! #TMJ
